

Homework 10 - Discussion

For this Ben Olschar, 108021211678 and Frederik Hüttemann, 108021215247 cooperated with each other. We did implement the code on our own, but agreed to using one version. The discussion and results are made in cooperation.

Prelims:

The execution was done on Google Colab using the Nvidia Tesla T4.

Task 1: Execution Time Measurements

To emulate processing of 100 images we reused the convolution kernel working in global memory. The image was set to be Full-HD with single float precision for each pixel. To check the needed time for processing of one image we run it in task 1. The following execution times have been observed:

- **Host-to-Device (H2D) transfer:** 1.06 ms
- **Kernel execution:** 2.03 ms
- **Device-to-Host (D2H) transfer:** 0.80 ms

As we can see, all parts of the execution take approximately the same time to execute.

Task 2: Sequential Processing (Single Stream)

Here, a single stream was used to process 100 images sequentially. Each image goes through the complete pipeline (H2D transfer, kernel execution, and D2H transfer) before new image is processed. This way we can also measure the average time for each part of the pipeline. We would expect, that H2D would take ~100 ms, the kernel itself ~200 ms and the D2H ~100 ms again for all images. We would therefore expect a overall runtime of ~400 to ~500 ms to completely process the 100 images.

The observed times match our expectations for the data transfer while the kernel execution takes longer than expected from our time measurements. In total, processing 100 images takes ~600 ms while the kernel execution takes the most time of ~440 ms. This is probably because for the time measurements, we ran the kernel multiple times so the results are amortized. Because the amortization couldnt be implemented for the streaming task, we also did not implement it for the sequential task. This way, both tasks are still comparable in the execution times.

Task 3: Kernel Processing with Streaming

The same kernel was run on 100 images again. This time, we used streaming / asynchronous optimization. Here, we can see a performance of 150 % com-

pared to the sequential run. The overall execution time for 100 images took 410 ms which is roughly 200 ms less than with a single stream. What could also be noticed is that the streaming is actually as fast as the sum of all parts of the pipeline. As already mentioned above, when summing the execution times for all parts and all images, we would expect a runtime of ~400 ms which also matches the observation from streaming.

In total, we can see that streaming leads to a huge performance increase compared to sequential kernel calls. This is because the operations are overlapped and the GPU is used more efficiently. While in sequential processing, the GPU is idle during data transfers, in streaming, data transfers and kernel executions are overlapped, leading to better utilization of the GPU resources and reduced overall execution time.